

1 Key Concepts

1.1 DP Dimensionality

Last week, we saw several examples of dynamic programming problems with a single variable of concern (index of a list, the length of a string, etc.). However, some problems often involve multiple independent variables representing the possible scenarios in which we can find ourselves. For example, a substring can be defined as the portion of the string between two indices i and j . The dimensionality of the recurrence affects how the recurrence will be defined, and how the table will be implemented.

1.2 Shortest-Path DP

In this course, we consider the following two *shortest-path problems* on graphs. In both problems, the input is a weighted directed graph $G = (V, E)$, with a “weight function” $w: E \rightarrow \mathbb{R}$ that maps each edge to a weight (or cost, or length). Although individual edge weights can be negative, we assume that there is **no negative-weight cycle** in G . So, if there is a path from s to t in G , then there is a path with minimum total weight (or length); we call this a *shortest* path from s to t . We define $d(s, t)$ to be the total weight (or length) of the edges in such a path, and define $d(s, t) = \infty$ if no such path exists.

- **Single-Source Shortest Paths (SSSP):** Given a “source” vertex $s \in V$, find a shortest path from s to each vertex $t \in V$. (Value version: given s , find $d(s, t)$ for every $t \in V$.)
- **All-Pairs Shortest Paths (APSP):** For *every* pair of vertices $s, t \in V$, find a shortest path from s to t . (Value version: find $d(s, t)$ for every $s, t \in V$.)

Key observations.

1. If a shortest path P from s to t goes through vertex v , then P must be a **shortest path from s to v** , followed by a **shortest path from v to t** .

This optimal-substructure property can be shown by a substitution argument. Namely: if there were a shorter path from s to v , then we could substitute it for the s -to- v part of P , which would yield an s -to- t path that is even shorter than P —but this would contradict the hypothesis that P is a shortest such path.

2. Since there is no negative-weight cycle in the graph, there is a shortest path from s to t with **no cycle** in it. So, when considering a shortest path (from any source to any destination), we can assume without loss of generality that the path does not have a cycle.

SSSP recurrence (Bellman-Ford). Define $d^{(i)}(s, v)$ to be the length of a shortest path from s to v that uses exactly i edges, or ∞ if there is no such path. Then

$$d(s, v) = \min_{0 \leq i < n} d^{(i)}(s, v)$$

for each $v \in V$. This is because, by [Item 2](#), if there is an s -to- v path at all, then there is a shortest such path having less than n edges (since any path with n or more edges must visit some vertex more than once, and hence have a cycle). So, to compute $d(s, v)$ it suffices to compute the values of $d^{(i)}(s, v)$ for $0 \leq i < n$.

Due to [Item 1](#), the $d^{(i)}$ functions satisfy the following recurrence:

$$d^{(0)}(s, v) = \begin{cases} 0 & \text{if } s = v \\ \infty & \text{if } s \neq v. \end{cases}$$

$$d^{(i)}(s, v) = \min_{(u,v) \in E} \left\{ d^{(i-1)}(s, u) + w(u, v) \right\}.$$

APSP Recurrence (Floyd-Warshall). Fix an arbitrary ordering of the vertices, as $V = \{v_1, v_2, \dots, v_n\}$. Then define $d^{[i]}(s, v)$ to be the length of a shortest path from s to v **whose internal vertices (if any) are in** $\{v_1, \dots, v_i\}$; or ∞ if there is no such path. Then

$$d(s, v) = d^{[n]}(s, v)$$

for each $v \in V$. This is because in any shortest s -to- v path, its internal vertices are in $V = \{v_1, \dots, v_n\}$. In other words, the restriction on the internal vertices imposed by $d^{[i]}$ is not a restriction at all when $i = n$.

Due to [Items 1](#) and [2](#), the $d^{[i]}$ functions satisfy the following recurrence:

$$d^{[0]}(s, t) = \begin{cases} 0 & \text{if } s = t \\ w(s, t) & \text{if } (s, t) \in E \\ \infty & \text{otherwise.} \end{cases}$$

$$d^{[i]}(s, t) = \min \left\{ d^{[i-1]}(s, t), d^{[i-1]}(s, v_i) + d^{[i-1]}(v_i, t) \right\}.$$

1.3 Greedy Algorithms

A *greedy algorithm* computes a solution to an optimization problem by making (and committing to) a sequence of locally optimal choices. In general, there is no guarantee that such a sequence of locally optimal choices produces a global optimum. However, for some specific problems and greedy algorithms, we can prove that the result is indeed globally optimal.

Exchange arguments. A standard strategy for proving that a greedy algorithm correctly solves a particular optimization problem proceeds by an *exchange argument* (like the ones we gave for unweighted task selection, and for minimum spanning trees). Such an argument typically aims to show that the algorithm's output is optimal by first proving it is a subset of some optimal solution, then arguing they must be the same using the greedy algorithm's condition.

To show that the algorithm's output is a subset of some optimal solution, we proceed as follows:

1. Let $\text{ALG} = \{s_1, s_2, \dots, s_m\}$ denote the algorithm's final —i.e., its full ordered sequence of selected objects—and let OPT be some *arbitrary* optimal solution.
2. If ALG happens to be a subset of OPT , then we have reached the desired conclusion, and there is nothing more to prove. So, suppose otherwise, and let s_{diff} denote the first object selected by the algorithm that is not in OPT . (So, all the objects $s_1, \dots, s_{\text{diff}-1}$ that the algorithm selected prior to s_{diff} are in OPT .)
3. We then modify OPT into a different optimal solution OPT' that contains all of $s_1, \dots, s_{\text{diff}}$. We do this by identifying a suitable element $f \in \text{OPT}$ (where $f \notin \{s_1, s_2, \dots, s_{\text{diff}}\}$) and exchanging it with s_{diff} , defining $\text{OPT}' = \text{OPT} \cup \{s_{\text{diff}}\} \setminus \{f\}$.

4. We must choose the exchanged element f so that OPT' is also an optimal solution, i.e., it is valid (it meets all the constraints imposed by the problem), and it is optimal. The latter often follows because s_{diff} is “no worse than” f in terms of what we want to optimize. Arguing the former typically depends on the specifics of the problem and how f is defined, but it often uses the fact that s_{diff} is “compatible” with $s_1, \dots, s_{\text{diff}-1}$ (because all of them are in ALG).
5. Altogether, the exchange step shows that since the first $\text{diff} - 1$ selected objects from ALG belong to some optimal solution, then the first diff selected objects from ALG also belong to some (different) optimal solution. So, by repeated application of the exchange step (formally, using induction), we can conclude that the algorithm’s entire output ALG is a subset of some optimal solution.

Finally, it just remains to show that ALG is equal to (and not just a subset of) an optimal solution. This is typically done by arguing that ALG is maximal, i.e., that no more objects can be added to it without violating the constraints of the problem. So, ALG itself must be optimal.

2 Discussion Problems

1. Longest Palindromic Subsequence

Give a recurrence relation (including base cases) that is suitable for dynamic programming solutions to the following problem. You do not need to prove your correctness.

Given a string S , determine the length of a longest subsequence of S (the characters do not need to be consecutive) that is a palindrome. Recall that a palindrome is a string which is the same forwards and backwards (e.g., “racecar”).

Solution: For an input string $S[1 \dots n]$, define the function $\text{LPS}[i, j]$ for $1 \leq i \leq j \leq n$ to be the length of a longest palindromic subsequence of $S[i \dots j]$. This indicates that we are only interested in subproblems that correspond to substrings of S . Clearly, $S[1, n]$ is the length of a longest palindromic subsequence of all of S .

We have the following base cases: $\text{LPS}[i, i] = 1$ for all i , because a single character is a palindrome, and $\text{LPS}[i, j] = 0$ when $i > j$.

For the recursive case, if $S[i] = S[j]$, then a longest palindromic subsequence (l.p.s.) of $S[i \dots j]$ includes those two characters at the beginning and end, with an l.p.s. of $S[i+1, j-1]$ between them.

Otherwise, an l.p.s. of $S[i \dots j]$ can’t include both of $S[i]$ and $S[j]$ (it wouldn’t be a palindrome), so it is the longer of an l.p.s. of $S[i+1 \dots j]$ and of $S[i \dots j-1]$.

All this yields the recurrence

$$\text{LPS}[i, j] = \begin{cases} 2 + \text{LPS}[i+1, j-1] & \text{if } S[i] = S[j] \\ \max\{\text{LPS}[i+1, j], \text{LPS}[i, j-1]\} & \text{otherwise.} \end{cases}$$

2. Bellman-Ford and Floyd-Warshall.

Select **all the true statements**.

- ✓ If p is the *unique* shortest s -to- t path in a graph—i.e., every other s -to- t path has strictly larger length—and v is a vertex on p , then the s -to- v portion of p is the *unique* shortest s -to- v path in the graph.
- ✓ In the Floyd–Warshall recurrence, the values $d^{[i]}(s, t)$ are *non-increasing* as i increases, i.e., $d^{[j]}(s, t) \leq d^{[i]}(s, t)$ for all $j \geq i$ and all vertices s, t .
- If a graph has no negative-weight cycle, then no shortest path in the graph contains a cycle.
- ✓ To fill each individual entry of the DP table in the Floyd–Warshall algorithm, we need to look at only a constant number of cells, whereas this may not be the case in the Bellman–Ford algorithm.

Solution:

- This is true, which we can see by a substitution argument: if there was a different shortest s -to- v path, we could substitute it for the s -to- v portion of p . This would yield a different s -to- t path having the same length as p , which would therefore also be a shortest s -to- t path. But this would violate the assumed uniqueness of p .
- This can be seen directly from the Floyd–Warshall recurrence

$$d^{[i+1]}(s, t) = \min\{d^{[i]}(s, t), d^{[i]}(s, i+1) + d^{[i]}(i+1, t)\} \leq d^{[i]}(s, t).$$

Alternatively, we recall from the definition of $d^{[i]}(s, t)$ that it is the length of a shortest s -to- t path whose intermediate vertices are a subset of $\{1, \dots, i\}$. Replacing i with any $j \geq i$ is a more permissive constraint, which cannot make the shortest path length larger.

- If there is a *weight-zero* cycle (which the hypothesis allows), then any path that goes fully around the cycle at least once is a shortest path from a vertex to itself that has a cycle.

We remark that a similar conclusion is true, and was covered in class: for any s, t there *exists* a shortest s -to- t path with no cycle (if there is an s -to- t path at all). However, the above example shows that this is not true of *all* shortest paths.

- This can be seen directly from the recurrence or pseudocode of the algorithms: FW looks at just 3 other cells (per filled cell), while BF looks at one cell for each of the target vertex’s incoming edges.

3. Vertex-weighted single-source lowest-cost paths (VWSSLCP).

In this problem we consider a variation of shortest-paths DP. Suppose we are given a graph $G = (V, E)$ and a cost function $c: V \rightarrow \mathbb{R}$ that maps each *vertex* to a cost. The cost of a path is simply the total cost of all the vertices on the path (including the starting and ending vertex). Again, we assume that there is **no negative-cost cycle** (although individual vertex costs can still be negative).

Given a source vertex $s \in V$, our goal is to find the *minimum path cost* from s to each vertex $v \in V$. Give a recurrence relation for this problem. Remember to include appropriate base cases(s).

Solution: We notice that the key observations from [Section 1.2](#) for shortest paths *still apply* to lowest-cost paths. We can use the same insights behind the Bellman-Ford recurrence in the lowest-cost path recurrence.

We give our recurrence by modifying Bellman-Ford. Similarly, we define $d^{(i)}(s, v)$ to be the cost of a lowest-cost path from s to v that uses **exactly** i edges, or ∞ if there is no such path. Then

$$d^{(0)}(s, v) = \begin{cases} c(s) & \text{if } s = v \\ \infty & \text{if } s \neq v, \end{cases}$$

$$d^{(i)}(s, v) = \min_{(u,v) \in E} \{d^{(i-1)}(s, u) + c(v)\}.$$

For the base case (i.e., lowest-cost paths with 0 edges), if $s = v$, a path that starts and ends at the same node s has cost $c(s)$. Otherwise, we cannot go from s to v using 0 edges, so the cost is ∞ .

In the recurrence (lowest-cost paths with exactly i edges), we know that to arrive at v , we must have come from some immediate predecessor vertex u , where $(u, v) \in E$. And by definition, the lowest cost to go from s to u using $i - 1$ edges is $d^{(i-1)}(s, u)$. So, the lowest-cost s -to- v path that uses exactly i edges is given by the minimum over all these options, which is captured in the above recurrence.

4. Copying homework problems.

You are copying n problems from your textbook to the sheets of paper you do your homework on. You need to copy all the problems, preserving their order. The problems fill various numbers of lines, and each sheet of paper has the same number of lines on it. Each problem must appear on a single sheet of paper; it can't be split across more than one sheet.

- (a) Describe a greedy algorithm to minimize the number of pages you need.

Solution: The greedy algorithm is to put as many problems as possible onto the page, until the next problem would exceed the line limit. Then start a new page and repeat with the remaining problems, until all the problems are copied.

- (b) Prove that the proposed greedy algorithm is correct. In other words, prove that the proposed algorithm uses the minimum number of pages, while satisfying the constraints given in the problem.

Solution: It is clear that the algorithm satisfies the constraints, because it does not ever split a problem over multiple sheets, or overfill any page. So, we just need to show it minimizes the number of pages.

We'll give an exchange argument to show this. Let $\text{ALG} = p_1, \dots, p_n$ be the non-decreasing sequence of page numbers on which our algorithm places the problems, where problem i appears on page p_i . And let $\text{OPT} = q_1, \dots, q_n$ be an arbitrary *optimal* solution, i.e., a non-decreasing sequence of page numbers where problem i appears on page q_i , and q_n is minimized. If $\text{ALG} = \text{OPT}$, then ALG is optimal, and there is nothing more to analyze.

So suppose that $\text{ALG} \neq \text{OPT}$, and consider the *first* problem i that appears on a different page in ALG than in OPT; that is, $p_j = q_j$ for all $j < i$, but $p_i \neq q_i$. Then $p_i < q_i$ by the design of ALG. We construct a different optimal solution OPT' that starts with $p_1, \dots, p_{i-1}, p_i$, i.e., it agrees with ALG on the first i problems (instead of just the first $i - 1$ problems, as OPT does). Specifically, we construct OPT' from OPT simply by replacing q_i with p_i , i.e., OPT' places problem i on the earlier page p_i rather than on page q_i . (Otherwise, OPT' is identical to OPT.)

We need to show that OPT' is a valid solution, and that it is optimal. For validity, notice that OPT' does not “overfill” page p_i , because ALG fits all the problems on page p_i that OPT' does, and possibly more. (This uses the fact that $q_j > p_i$ for all $j > i$, since $q_j \geq q_i > p_i$ by the non-decreasing property of OPT.) Moreover, no other page gets any additional problem(s) in OPT' versus OPT, so OPT' is indeed valid. For optimality, note that OPT' does not increase the page number of the final problem n (or any problem), so it does not use more pages than OPT does. Since OPT is optimal, so is OPT' .

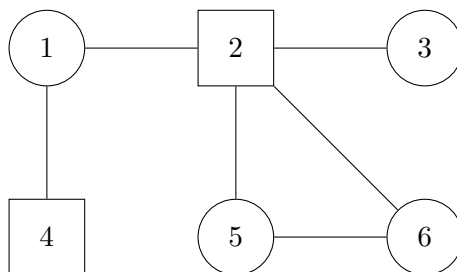
Finally, by repeating the exchange step, we see that ALG agrees with some optimal solution for *all* n problems, hence ALG itself is optimal.

5. Dominating Set.

A *dominating set* S in a graph G is a set of vertices for which every vertex of G either is in S , or is adjacent to some vertex in S .

We are interested in a *smallest* dominating set of a given graph, i.e., one that has the fewest possible vertices. (There may be more than one smallest dominating set.)

For example, the following graph has a smallest dominating set $S^* = \{2, 4\}$: every vertex other than 2 and 4 is adjacent to 2 or 4 (or both), and there is no dominating set consisting of a single vertex.



Consider the following greedy algorithm for finding a dominating set in a graph.

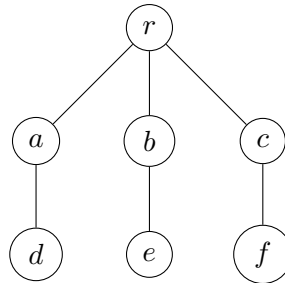
```

1: function GREEDYDS( $G$ )
2:    $S \leftarrow \emptyset$ 
3:   while  $G$  has at least one vertex do
4:     Select any vertex  $v$  in  $G$  that has largest degree (i.e., the most neighbors)
5:     Add  $v$  to  $S$ 
6:     Remove  $v$  and all its neighbors, including all incident edges, from  $G$ 
7:   return  $S$ 

```

Give a small graph G on which the algorithm **might not return a *smallest dominating set***. Specifically, **give a sequence of vertices that the algorithm might choose** to make up its final output set, and **give an optimal dominating set of G that is smaller** than this output set.

Solution: There are many different graphs on which this algorithm does not (necessarily) give a smallest dominating set. Here is one (which does not even rely on the algorithm ‘breaking ties’ in a certain way when choosing vertices v):



A smallest dominating set is $\{a, b, c\}$. By inspection, no single vertex forms a dominating set of the graph. No set of two vertices works either, because at least one vertex from each pair (a, d) , (b, e) , and (c, f) must be taken to form a dominating set.

The given algorithm will first add r to S , because it is the unique vertex with largest degree (3). After this, the only vertices left in the graph are d, e, f , and the algorithm takes all three of them because no edges remain. So the algorithm outputs $\{r, d, e, f\}$ as its dominating set, which is not optimal.

3 Additional Problems

1. Matrix-chain multiplication

Given a sequence of matrices $(M[1, \dots, n])$ to multiply together, determine the minimum number of element-element multiplications required (you may disregard additions).

As an example, suppose the sequence of matrices to multiply is ABC , where A is 2×3 , B is 3×4 , and C is 4×5 .

Computing $(AB)C$ requires $2 \cdot 3 \cdot 4 + 2 \cdot 4 \cdot 5 = 64$ multiplications. Computing $A(BC)$ requires $3 \cdot 4 \cdot 5 + 2 \cdot 3 \cdot 5 = 90$ multiplications. Therefore, the minimum number of multiplications

required is 64.

The following is a short example of how to perform matrix multiplication:

$$\text{If } A = \begin{bmatrix} a_{00} & a_{01} \\ a_{10} & a_{11} \end{bmatrix} \text{ and } B = \begin{bmatrix} b_{00} & b_{01} \\ b_{10} & b_{11} \end{bmatrix}, \text{ then } AB = \begin{bmatrix} a_{00}b_{00} + a_{01}b_{10} & a_{00}b_{01} + a_{01}b_{11} \\ a_{10}b_{00} + a_{11}b_{10} & a_{10}b_{01} + a_{11}b_{11} \end{bmatrix}.$$

Remarks: Matrix multiplication is associative. Given two matrices with dimensions $m \times n$ and $n \times p$, we require mnp multiplications because the resulting matrix is of dimension $m \times p$ and each element requires n multiplications. You can read more about matrix multiplication here: https://en.wikipedia.org/wiki/Matrix_multiplication

Solution: Let us define $L(i, j)$ to be the minimum number of multiplications required to multiply the i^{th} through j^{th} matrices. The base cases are when we only have a single matrix, i.e. $i = j$, in which case no multiplications are required: $L(i, i) = 0$ for each i .

Let each $M[i]$ be of the form (r, c) , where r is the number of rows and c is the number of columns in the matrix. Then

$$L(i, j) = \min_{i \leq k < j} (L(i, k) + L(k + 1, j) + M[i].r \cdot M[k].c \cdot M[j].c).$$

This recurrence will find the minimum by recognizing that each sequence of matrices that are multiplied have a final multiplication, therefore, by checking each final breakpoint, we can find the breakpoint that corresponds to the minimum number of multiplications.

2. Scooby-Doo escape room.

Scooby-Doo is stuck in an $n \times n$ -sized escape room; the “northwest” corner of the room is denoted $(1, 1)$. Each 1×1 square in this room has a certain amount of Scooby-Snacks. You are given the matrix H , where $H[i][j] := H_{ij}$ contains the number of Scooby-Snacks in the (i, j) th square of the room.

Scooby wants to maximize the number of Scooby Snacks he eats while escaping from a monster coming from the north side of the room. If he is at square (i, j) , then from his current location he can move only to squares $(i + 1, j)$, $(i + 1, j + 1)$ or $(i + 1, j - 1)$. (In other words, he can only move directly south, southeast, or southwest.) Otherwise, the monster will catch up and eat him!

Scooby escapes when he reaches the south wall i.e. a location of the form $(n, \star)$ (in other words when Scooby gets to the other end of the room). Scooby can start from any square $(1, \star)$ in the northernmost row.

- Give a recurrence relation (including base cases), that would yield an $O(n^2)$ time dynamic programming solution to the problem. Briefly explain why your recurrence relation is correct.

Solution: Because the monster has given Scooby-Doo enough freedom to choose any position he likes from the first row to start at, it will be helpful to define a recurrence

$MSS(i, j)$ which returns the maximum number of Scooby-snacks he can eat given that he starts at position (i, j) and he can only move directly southwest, south or southeast from his current position.

We start from the base case. If at any point $i = n$, Scooby cannot move anywhere from this position because he has reached the south wall of the room and as a result $MSS(i, j) = H[i, j]$. This basically means that he can only eat the Scooby-Snacks that are at his current position.

We now consider the recursive case. If Scooby is at position (i, j) for $i < n$, then Scooby has not reached the south wall of the room, and has at most three possible moves (1) southwest, (2) south, and (3) southeast. Upon choosing a move, Scooby will be able to eat all the Scooby-Snacks at the corresponding position, and, to maximize the overall number of snack he eats during the escape, Scooby will need to eat the maximum possible number of Scooby-Snacks from that position. The moves are limited only if Scooby is along either the westernmost wall (at a position of the form $(i, 1)$) or the easternmost wall (at a position of the form (i, n)).

Let SW denote the maximum number of Scooby-Snacks he can eat if his initial position is $(i + 1, j - 1)$, S be the same when his initial position is $(i + 1, j)$ and SE be the same when his initial position is $(i + 1, j + 1)$ i.e.,

$$SW = MSS(i + 1, j - 1); S = MSS(i + 1, j); SE = MSS(i + 1, j + 1).$$

If $j = 1$, there is no south-west square to move to so we obtain the recurrence

$$MSS(i, j) = H[i, j] + \max\{S, SE\}.$$

If $j = n$, there is no south-east square to move to so we obtain the recurrence

$$MSS(i, j) = H[i, j] + \max\{S, SW\}.$$

Hence, $MSS(i, j)$ satisfies the recurrence

$$MSS[i, j, H] = \begin{cases} H[i][j] & \text{if } i = n, \\ H[i][j] + \max\{SE, S\} & \text{else if } j = 1, \\ H[i][j] + \max\{SW, S\} & \text{else if } j = n, \\ H[i][j] + \max\{SW, S, SE\} & \text{otherwise.} \end{cases}$$

- (b) Give pseudocode implementing an $O(n^2)$ dynamic programming algorithm MAX-SCOOBY-SNACK that computes the maximum number of Scooby Snacks that Scooby can eat during his escape under the problem constraints.

Solution: As Scooby can pick any square in the top row to begin with, we can compute $MSS[i, j, H]$ for $1 \leq j \leq n$ and return the maximum quantity. Hence:

$$\text{MAX-SCOOBY-SNACK}(H) = \max_{1 \leq j \leq n} MSS(1, j)$$

To implement this strategy efficiently, we can apply dynamic programming to the computation of MSS.

Input: Snack matrix H and a *global* $n \times n$ memo which will store MSS values

Output: MAX-SCOOPY-SNACK(H)

```
1: function MAX-SCOOPY-SNACK( $H[1 \dots n][1 \dots n]$ )
2:   for  $i = n, \dots, 1$  do // It is important that this loop goes from  $n$  to 1
3:     for  $j = 1, \dots, n$  do
4:       if  $i = n$  then
5:          $\text{MEMO}[i][j] \leftarrow H[i][j]$ 
6:       else if  $j = 1$  then
7:          $\text{MEMO}[i][j] \leftarrow H[i][j] + \max(\text{MEMO}[i + 1][j], \text{MEMO}[i + 1][j + 1])$ 
8:       else if  $j = n$  then
9:          $\text{MEMO}[i][j] \leftarrow H[i][j] + \max(\text{MEMO}[i + 1][j - 1], \text{MEMO}[i + 1][j])$ 
10:      else
11:         $\text{MEMO}[i][j] \leftarrow H[i][j] + \max(\text{MEMO}[i + 1][j - 1], \text{MEMO}[i + 1][j], \text{MEMO}[i + 1][j + 1])$ 
12:       $\text{result} \leftarrow \max_{1 \leq i \leq n}(\text{MEMO}[1][i])$ 
13:      return  $\text{result}$ 
```